

ml-lab03-sb

September 28, 2024

Name : Soham Bhattacharya ; ID : B2430059 ; BDA 24-46 ; Lab 03 Assignment

1 3 Random MNIST Images with Labels

1.1 Generating 3 random indices

```
[ ]: import random as rand
x1 = rand.randint(0,60000)
x2 = rand.randint(0,60000)
x3 = rand.randint(0,60000)

new_x = [x1, x2, x3]
```

```
[ ]: new_x
```

```
[ ]: [12355, 33191, 59025]
```

```
[ ]: import matplotlib.pyplot as plt
from keras.datasets import mnist

# Load the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
[ ]: X_train.shape
```

```
[ ]: (60000, 28, 28)
```

```
[ ]: X_train[59999]
```

```
[ ]: array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
           0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
           0,  0],
         [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
           0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
           0,  0],
         [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
           0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
           0,  0],
```

```

[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0, 0, 38, 48, 48, 22, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  62, 97, 198, 243, 254, 254, 212, 27, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 67,
  172, 254, 254, 225, 218, 218, 237, 248, 40, 0, 21, 164, 187,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 89, 219,
  254, 97, 67, 14, 0, 0, 92, 231, 122, 23, 203, 236, 59,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 25, 217, 242,
  92, 4, 0, 0, 0, 0, 4, 147, 253, 240, 232, 92, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 101, 255, 92,
  0, 0, 0, 0, 0, 0, 105, 254, 254, 177, 11, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 167, 244, 41,
  0, 0, 0, 7, 76, 199, 238, 239, 94, 10, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 192, 121, 0,
  0, 2, 63, 180, 254, 233, 126, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 190, 196, 14,
  2, 97, 254, 252, 146, 52, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 130, 225, 71,
  180, 232, 181, 60, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 130, 254, 254,
  230, 46, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 6, 77, 244, 254, 162,
  4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 110, 254, 218, 254, 116,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0]

```

```

0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 131, 254, 154, 28, 213, 86,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 66, 209, 153, 19, 19, 233, 60,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 142, 254, 165, 0, 14, 216, 167,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 90, 254, 175, 0, 18, 229, 92,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 26, 229, 249, 176, 222, 244, 44,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 73, 193, 197, 134, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0]], dtype=uint8)

```

1.2 Printing 3 digits placed at those randomly generated indices in the same plot

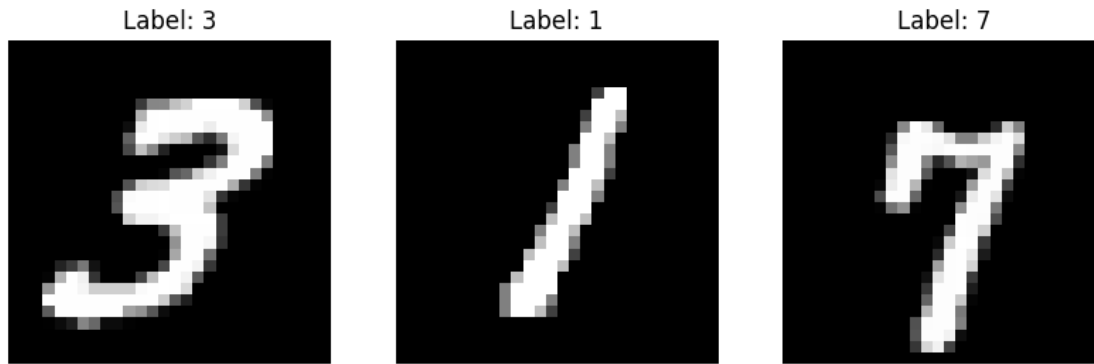
```

[ ]: # Set up the figure for subplots
fig, axes = plt.subplots(1, 3, figsize=(10, 4))

j=0
# Loop to display three different images
for i in new_x:
    axes[j].imshow(X_train[i], cmap='gray') # Show the image in grayscale
    axes[j].set_title(f"Label: {y_train[i]}") # Display the label
    axes[j].axis('off') # Turn off the axis
    j+=1

plt.show()

```



```
[ ]: import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

dig1 = X_train[x1]
dig2 = X_train[x2]
dig3 = X_train[x3]
plt.show()
```

1.3 Individually plotting the digits

```
[ ]: import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")
```

```
[ ]: # Individual Plots

# First entry
some_digit = X[x1]
plot_digit(some_digit)

plt.show()

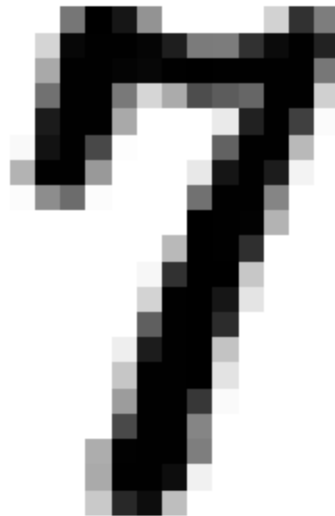
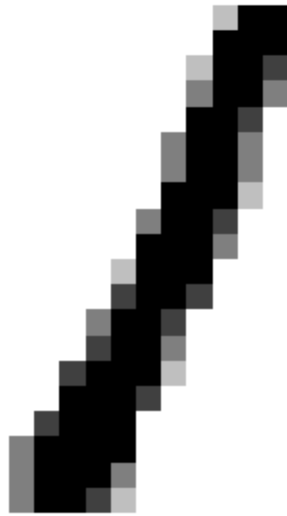
# Second entry
some_digit = X[x2]
plot_digit(some_digit)
```

```
plt.show()

# Third entry
some_digit = X[x3]
plot_digit(some_digit)

plt.show()
```





2 Binary Classification : Class 8 vs Rest

```
[ ]: from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)

[ ]: X, y = mnist.data, mnist.target
X
[ ]: array([[0, 0, 0, ..., 0, 0, 0],
          [0, 0, 0, ..., 0, 0, 0],
          [0, 0, 0, ..., 0, 0, 0],
          ...,
          [0, 0, 0, ..., 0, 0, 0],
          [0, 0, 0, ..., 0, 0, 0],
          [0, 0, 0, ..., 0, 0, 0]])

[ ]: X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

2.1 Training a Binary Classifier

```
[ ]: y_train_8 = (y_train == '8') # True for all 8s, False for all other digits
y_test_8 = (y_test == '8')

[ ]: from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_8)

[ ]: SGDClassifier(random_state=42)

[ ]: some_digit = X[x1]
# print(f'Some Digit = {some_digit}')
sgd_clf.predict([some_digit])

[ ]: array([False])
```

3 Classification Score

3.1 Measuring Accuracy Using Cross-Validation

```
[ ]: from sklearn.model_selection import cross_val_score

cross_val_score(sgd_clf, X_train, y_train_8, cv=3, scoring="accuracy")

[ ]: array([0.909 , 0.9286 , 0.88285])
```

4 Confusion Matrix

```
[ ]: from sklearn.model_selection import cross_val_predict
```

```
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_8, cv=3)
```

```
[ ]: from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_train_8, y_train_pred)
cm
```

```
[ ]: array([[50006,  4143],
          [ 1448, 4403]])
```

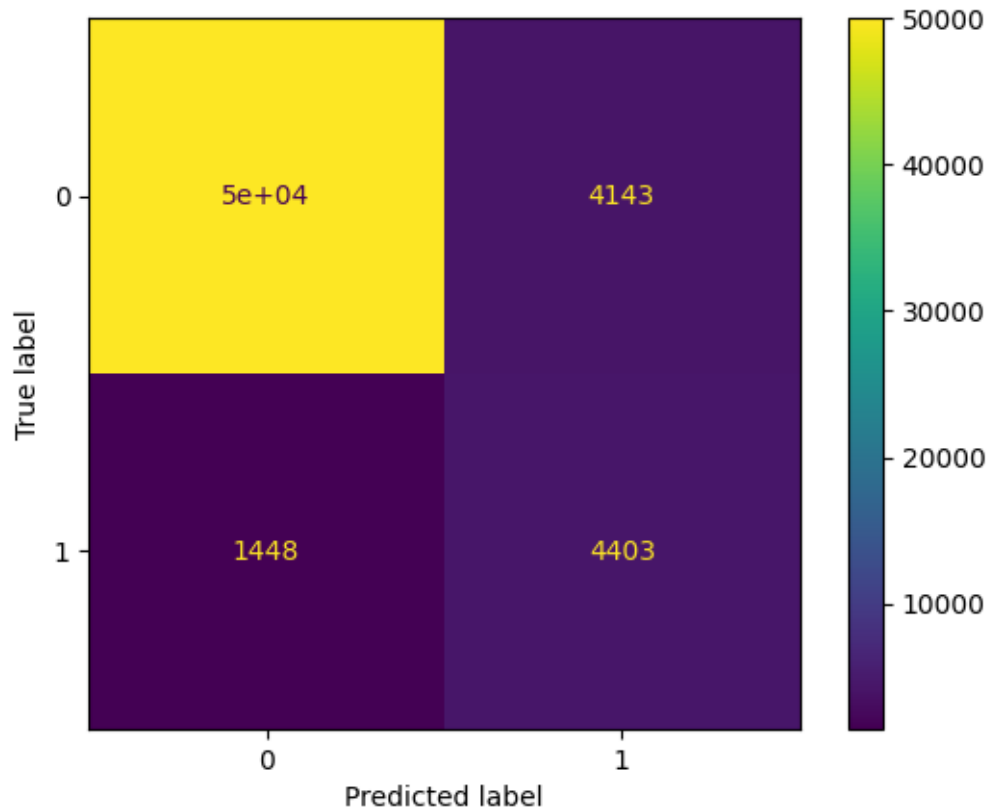
```
[ ]: y_train_perfect_predictions = y_train_8 # pretend we reached perfection
confusion_matrix(y_train_8, y_train_perfect_predictions)
```

```
[ ]: array([[54149,    0],
          [    0, 5851]])
```

```
[169]: from sklearn.metrics import ConfusionMatrixDisplay
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

```
[169]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x7838dc6bc910>
```

5 Precision

```
[ ]: from sklearn.metrics import precision_score, recall_score

precision_score(y_train_8, y_train_pred)
```

```
[ ]: 0.515211794991809
```

```
[ ]: # extra code - this cell also computes the precision: TP / (FP + TP)
cm[1, 1] / (cm[0, 1] + cm[1, 1])
```

```
[ ]: 0.515211794991809
```

6 Recall

```
[ ]: recall_score(y_train_8, y_train_pred)
```

```
[ ]: 0.7525209365920356
```

```
[ ]: # extra code - this cell also computes the recall: TP / (FN + TP)
cm[1, 1] / (cm[1, 0] + cm[1, 1])
```

```
[ ]: 0.7525209365920356
```

7 F1-Score

```
[ ]: from sklearn.metrics import f1_score

f1_score(y_train_8, y_train_pred)
```

```
[ ]: 0.6116552059456831
```

```
[ ]: # extra code - this cell also computes the f1 score
cm[1, 1] / (cm[1, 1] + (cm[1, 0] + cm[0, 1]) / 2)
```

```
[ ]: 0.6116552059456831
```

8 ROC-AUC and Trade-offs

8.1 Precision/Recall Trade-off

```
[ ]: y_scores = sgd_clf.decision_function([some_digit])
y_scores
```

```
[ ]: array([-7999.82404651])
```

```
[ ]: threshold = 0
y_some_digit_pred = (y_scores > threshold)
```

```
[ ]: y_some_digit_pred
```

```
[ ]: array([False])
```

```
[ ]: # extra code - just shows that y_scores > 0 produces the same result as
#           calling predict()
y_scores > 0
```

```
[ ]: array([False])
```

```
[ ]: threshold = 3000
y_some_digit_pred = (y_scores > threshold)
y_some_digit_pred
```

```
[ ]: array([False])
```

```
[ ]: y_scores = cross_val_predict(sgd_clf, X_train, y_train_8, cv=3,
    ↪method="decision_function")

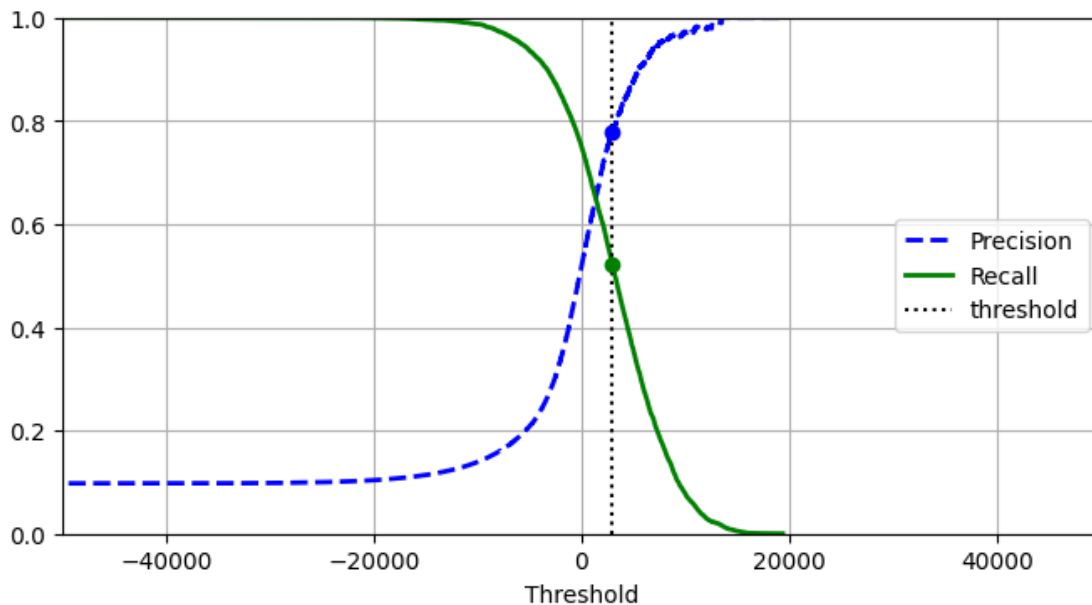
[ ]: from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_8, y_scores)

[ ]: plt.figure(figsize=(8, 4)) # extra code - it's not needed, just formatting
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")

# extra code - this section just beautifies and saves Figure 3-5
idx = (thresholds >= threshold).argmax() # first index threshold
plt.plot(thresholds[idx], precisions[idx], "bo")
plt.plot(thresholds[idx], recalls[idx], "go")
plt.axis([-50000, 50000, 0, 1])
plt.grid()
plt.xlabel("Threshold")
plt.legend(loc="center right")
# save_fig("precision_recall_vs_threshold_plot")

plt.show()
```



```
[ ]: import matplotlib.patches as patches # extra code - for the curved arrow
```

```

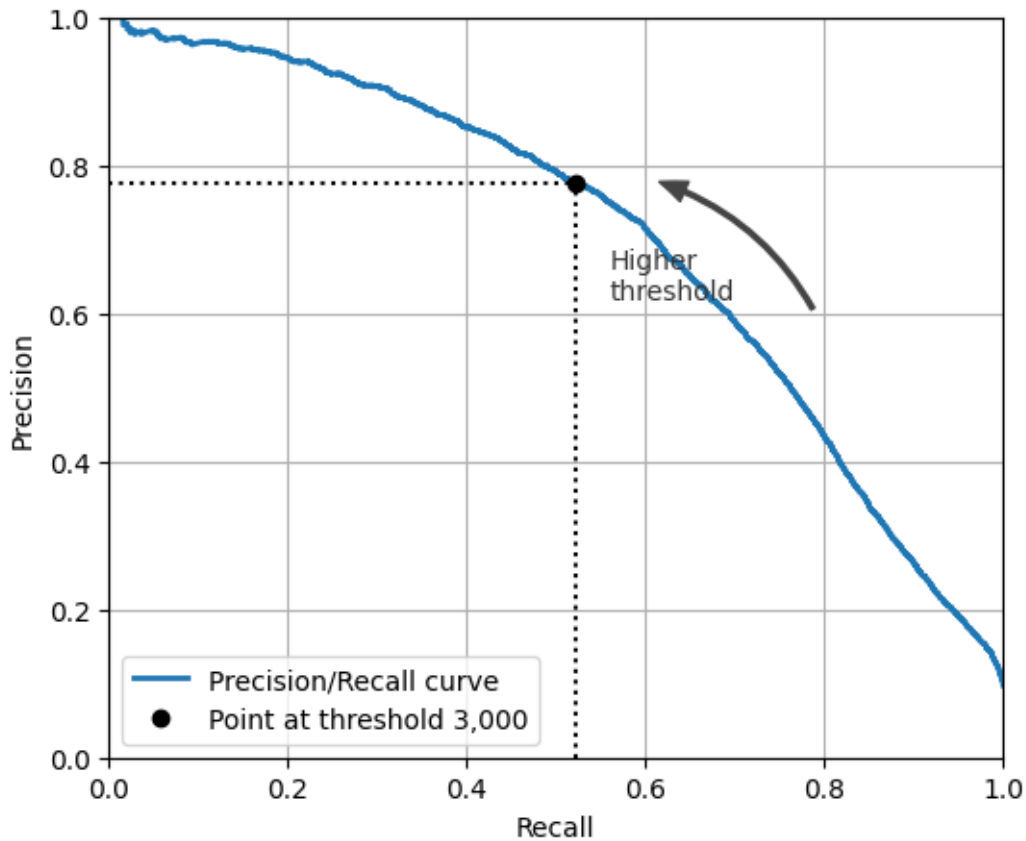
plt.figure(figsize=(6, 5)) # extra code - not needed, just formatting

plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")

# extra code - just beautifies and saves Figure 3-6
plt.plot([recalls[idx], recalls[idx]], [0., precisions[idx]], "k:")
plt.plot([0.0, recalls[idx]], [precisions[idx], precisions[idx]], "k:")
plt.plot([recalls[idx]], [precisions[idx]], "ko",
         label="Point at threshold 3,000")
plt.gca().add_patch(patches.FancyArrowPatch(
    (0.79, 0.60), (0.61, 0.78),
    connectionstyle="arc3,rad=.2",
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",
    color="#444444"))
plt.text(0.56, 0.62, "Higher\ntreshold", color="#333333")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.axis([0, 1, 0, 1])
plt.grid()
plt.legend(loc="lower left")
# save_fig("precision_vs_recall_plot")

plt.show()

```



```
[ ]: idx_for_90_precision = (precisions >= 0.90).argmax()
threshold_for_90_precision = thresholds[idx_for_90_precision]
threshold_for_90_precision
```

```
[ ]: 5558.688951079276
```

```
[ ]: y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

```
[ ]: precision_score(y_train_8, y_train_pred_90)
```

```
[ ]: 0.9003419638495359
```

```
[ ]: recall_at_90_precision = recall_score(y_train_8, y_train_pred_90)
recall_at_90_precision
```

```
[ ]: 0.3149888907878995
```

8.2 The ROC Curve

```
[ ]: from sklearn.metrics import roc_curve

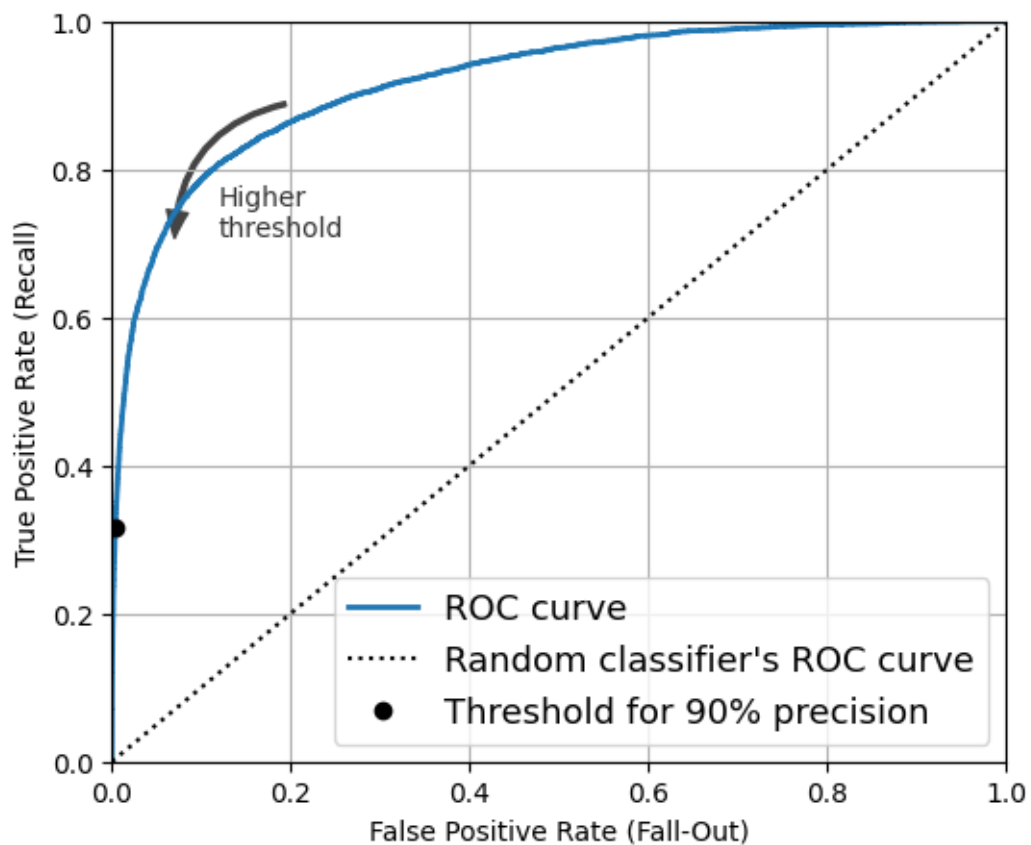
fpr, tpr, thresholds = roc_curve(y_train_8, y_scores)

[ ]: idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]

plt.figure(figsize=(6, 5)) # extra code - not needed, just formatting
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")

# extra code - just beautifies and saves Figure 3-7
plt.gca().add_patch(patches.FancyArrowPatch(
    (0.20, 0.89), (0.07, 0.70),
    connectionstyle="arc3,rad=.4",
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",
    color="#444444"))
plt.text(0.12, 0.71, "Higher\ntthreshold", color="#333333")
plt.xlabel('False Positive Rate (Fall-Out)')
plt.ylabel('True Positive Rate (Recall)')
plt.grid()
plt.axis([0, 1, 0, 1])
plt.legend(loc="lower right", fontsize=13)
# save_fig("roc_curve_plot")

plt.show()
```



```
[ ]: from sklearn.metrics import roc_auc_score  
  
roc_auc_score(y_train_8, y_scores)
```

```
[ ]: 0.9192599874103056
```